

Advanced Programming(CSBT215)

Library Management System using Python

ARITRA TALUKDAR

CSB24034

April 29, 2026

1 Introduction

This project demonstrates the design of a Library System using Object-Oriented Programming concepts in Python. The system models different types of library items such as books and DVDs using abstraction, inheritance, polymorphism, and method overriding.

2 Objectives

- To implement abstraction using an abstract base class
- To demonstrate inheritance and method overriding
- To implement polymorphism using a collection of objects
- To include an additional feature using a static class counter

3 Concepts Used

3.1 Abstraction

An abstract class `LibraryItem` is created using the `ABC` module. It defines common attributes like `title` and `year` and an abstract method `display_info()`.

3.2 Inheritance

Classes `Book` and `DVD` inherit from the base class `LibraryItem` and extend it with additional attributes.

3.3 Method Overriding

Both subclasses override the `display_info()` method to provide specific details.

3.4 Polymorphism

A list of `LibraryItem` objects is used to demonstrate polymorphism, where the same method call behaves differently for different objects.

3.5 Static/Class Variable

A class variable `item_count` is used to keep track of the total number of objects created.

4 System Design

- Base Class: `LibraryItem`
- Derived Classes:
 - `Book`
 - `DVD`

5 Implementation

5.1 Python Code

```
from abc import ABC, abstractmethod

# Abstract Base Class
class LibraryItem(ABC):
    item_count = 0    # static/class variable (extra feature)

    def __init__(self, title, year):
        self.title = title
        self.year = year
        LibraryItem.item_count += 1    # increment counter

    @abstractmethod
    def display_info(self):
        pass

# Book subclass
class Book(LibraryItem):
    def __init__(self, title, year, author):
        super().__init__(title, year)
        self.author = author

    def display_info(self):
        print(f"[Book] Title: {self.title}, Year: {self.year}, Author: {self.author}")
```

```

# DVD subclass
class DVD(LibraryItem):
    def __init__(self, title, year, duration, genre):
        super().__init__(title, year)
        self.duration = duration
        self.genre = genre

    def display_info(self):
        print(f"[DVD] Title: {self.title}, Year: {self.year},
              Duration: {self.duration} mins, Genre: {self.genre}")

# ---- Main Program (Polymorphism demonstration) ----

# Create objects
item1 = Book("The Alchemist", 1988, "Paulo Coelho")
item2 = DVD("Inception", 2010, 148, "Sci-Fi")
item3 = Book("Clean Code", 2008, "Robert C. Martin")

# Store in a common list (base class reference)
library_items = [item1, item2, item3]

# Polymorphism: same method, different behavior
for item in library_items:
    item.display_info()

# Show total items (static feature)
print("\nTotal Library Items:", LibraryItem.item_count)

```

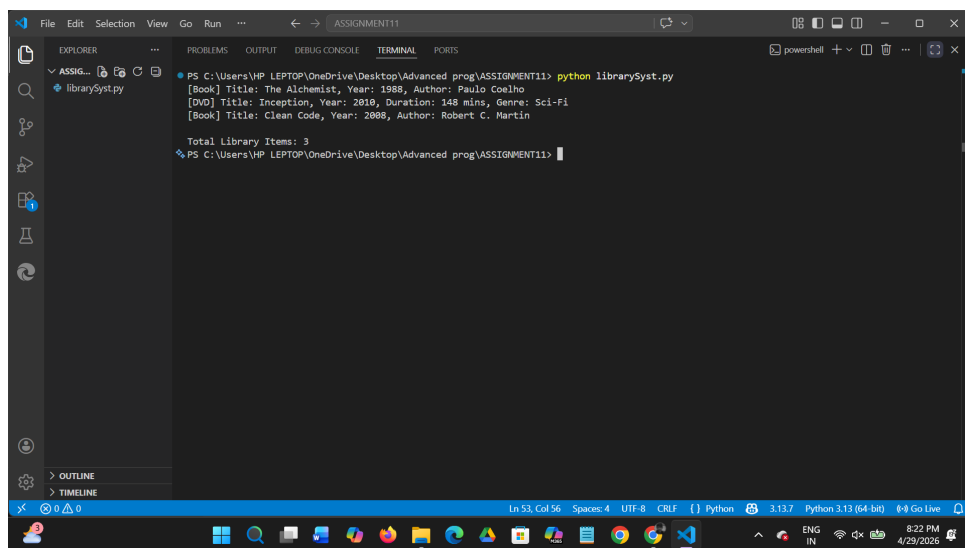
6 Advantages of the System

- Promotes code reusability through inheritance
- Ensures flexibility using polymorphism
- Clean and maintainable design using abstraction
- Easily extendable for new item types

7 Conclusion

The Library System successfully demonstrates key Object-Oriented Programming principles. The use of abstraction, inheritance, and polymorphism makes the system scalable and easy to maintain.

8 Output Screenshot



```
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT11> python librarySystem.py
[Book] Title: The Alchemist, Year: 1988, Author: Paulo Coelho
[Book] Title: Inception, Year: 2010, Duration: 148 mins, Genre: Sci-Fi
[Book] Title: Clean Code, Year: 2008, Author: Robert C. Martin

Total Library Items: 3
PS C:\Users\HP LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT11>
```

Figure 1: Sample Output of the Library System